

EXPERIMENT 2

(A)

A university administration requires an analytical performance report for each academic unit. Write a PostgreSQL solution to calculate the average marks scored by students grouped by their respective departments. The solution must join the student profile, course subjects, and grade marks tables, returning a breakdown sorted by the highest average marks down to the lowest.

CODE:

```
CREATE TABLE Departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50) NOT NULL);  
  
CREATE TABLE Students (  
    student_id INT PRIMARY KEY,  
    student_name VARCHAR(50) NOT NULL,  
    dept_id INT REFERENCES Departments(dept_id));  
  
CREATE TABLE Subjects (  
    subject_id INT PRIMARY KEY,  
    subject_name VARCHAR(50) NOT NULL);  
  
CREATE TABLE Marks (  
    student_id INT REFERENCES Students(student_id),  
    subject_id INT REFERENCES Subjects(subject_id),  
    marks_scored INT NOT NULL,  
    PRIMARY KEY (student_id, subject_id));  
  
INSERT INTO Departments (dept_id, dept_name) VALUES  
(1, 'Computer Science'),  
(2, 'Mathematics');  
  
INSERT INTO Students (student_id, student_name, dept_id) VALUES  
(101, 'Alex', 1),
```

24BAI70068

(102, 'Bella', 1),

(103, 'Charlie', 2);

INSERT INTO Subjects (subject_id, subject_name) VALUES

(501, 'Data Structures'),

(502, 'Calculus');

INSERT INTO Marks (student_id, subject_id, marks_scored) VALUES

(101, 501, 85),

(101, 502, 90),

(102, 501, 75),

(103, 502, 95);

SELECT D.DEPT_NAME AS DEPARTMENT,

AVG(M.MARKS_SCORED) AS "AVERAGE MARKS"

FROM DEPARTMENTS AS D

JOIN

STUDENTS AS S

ON

D.DEPT_ID = S.DEPT_ID

JOIN

MARKS AS M

ON

M.STUDENT_ID = S.STUDENT_ID

GROUP BY D.DEPT_NAME

ORDER BY "AVERAGE MARKS" DESC;

Showing rows: 1 to 2   Page No: 1		
	department character varying (50) 	AVERAGE MARKS numeric 
1	Mathematics	95.0000000000000000
2	Computer Science	83.3333333333333333

Two legacy HR systems (A and B) have separate records of employee salaries. These records may overlap. Management wants to **merge these datasets** and identify **each unique employee** (by EmpID) along with their **lowest recorded salary** across both systems.

Objective

1. Combine two tables A and B.
2. Return each EmpID with their **lowest salary**, and the corresponding **Ename**.

Table A

EmpID	Ename	Salary
1	AA	1000
2	BB	300

Table B

EmpID	Ename	Salary
2	BB	400
3	CC	100

CODE:

```
select empid,min(ename),min(salary)
from(
    select * from A Union all
    select * from B
) as intermediate_result
group by empid;
```

	empid integer	min text	min integer
1	3	CC	100
2	2	BB	300
3	1	AA	1000

(C)

A multinational organization maintains separate records for employee information and department details. The management wants to identify the employees who receive the highest salary within their respective departments for an annual performance report.

If more than one employee earns the same highest salary in a department, all such employees should be included in the report. The final output should display the **Department Name**, **Employee Name**, and **Salary**, arranged in ascending order of department name.

Note: The employee and department information are maintained in separate normalized tables.

CODE:

```
select E.NAME,  
       E.SALARY,  
       D.DEPT_NAME  
FROM  
EMPLOYEE AS E  
JOIN  
DEPARTMENT AS D  
ON  
E.department_id = D.ID  
WHERE E.SALARY IN  
(  
    SELECT MAX(E1.SALARY)  
    FROM EMPLOYEE AS E1  
    WHERE E1.department_id = E.department_id  
);
```

	name character varying (50) 🔒	salary integer 🔒	dept_name character varying (50) 🔒
1	JIM	90000	IT
2	HENRY	80000	SALES
3	MAX	90000	IT